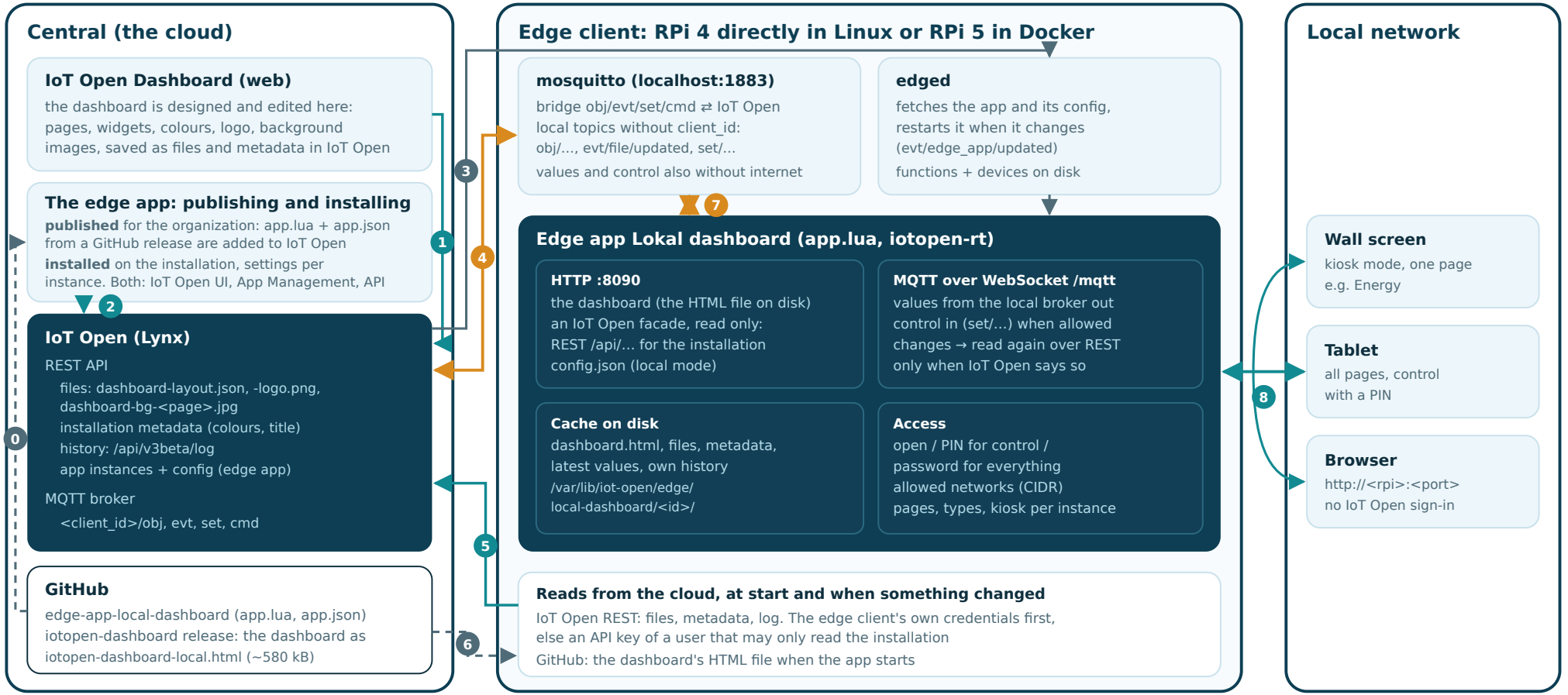
IoT Open Local Dashboard via edge-app

Designed in the cloud, shown locally by the edge client, also without internet



0 The edge app (app.lua and app.json from a GitHub release) is published for the organization in IoT Open. **1** The dashboard saves the layout and images as installation files, colours and title as metadata. **2** The app is installed with settings per instance. 0 and 2: IoT Open's user interface, App Management or the API. **3** edged restarts the app when its configuration changes.

4 The bridge for obj/evt/set/cmd between IoT Open and the local broker. **5** The app reads files, metadata and history over REST, only at start and when `evt/file/updated` or `evt/installation/updated` arrives. **6** The dashboard's HTML file is downloaded from the latest release when the app starts.

7 Live values from the local broker, and control back (`set/…`). **8** The screens load the dashboard and talk to the app as if it were IoT Open: REST and MQTT over WebSocket. Without internet, 7 and 8 go on with the cached layout and the app's own history.

Data, operation and security

What comes from where, and what works without internet

Where the data comes from

Data	Source	Without internet
The dashboard (the code)	GitHub release, downloaded at start	**saved copy**
Layout, pages, logo, background images	Installation files `dashboard-*` (REST)	**cache**
Colours, title	Installation metadata (REST)	**cache**
Functions and devices	The edge client's own copies	**yes**
Live values	The local broker (obj/...)	**yes**
Control (switches, set points)	Published on the local broker (set/...)	**yes**
History, graphs, consumption	IoT Open's v3beta log (REST)	**own history, 1-168 h**
Map (Tracker)	Map tiles from the internet	**hidden by default**

Signing in to IoT Open

- The app always tries the edge client's own credentials first. Today they may read functions, devices and latest values, but not files, metadata or the log.
- Until they may, an optional **API key** in the app's settings is used. The key should belong to a user of its own that may only read the installation. It is only used in the app's own requests and is never sent to the browser.
- When the edge client is given read access, the key can be removed without a new version of the app.

Two ways to run the edge client, the same app

	Directly in Linux, e.g. RPi 4	In Docker, e.g. RPi 5 with App Management
Reached on	`http://<rpi>:8090` directly	A published port, e.g. `http://<rpi>:18010`
Port	Open 8090 towards the LAN in the Pi's firewall, if it has one	*Edge client → Publicerade portar* in App Management (`EDGE_PORT_RANGE` , `EDGE_PORT_BIND`)
The app's data	On the SD card or disk	The volume `iotopen-edge-<id>-data` , kept on update
Memory	The whole Pi	The container has 256 MB, plenty
Check first	Run `tools/check.lua` on the edge client (tested on a Raspberry Pi 4)	Tested in IoT Open's edge client image

Access and security

- **Open**: everyone on the network can see and control. **PIN**: everyone can see, *Lås upp* in the header gives control for 12 h. **Password**: everything needs the password (kept in the browser for 30 days).
- *Allowed networks* limit who can reach the app. Five wrong attempts lock out the address for five minutes.
- The app answers on plain http: only on a network you trust, never towards the internet. Changes are always made in the cloud; locally the dashboard is read only.

Per instance (several screens)

Each instance has its own pages, start page, hidden widget types (the map by default), kiosk mode, port and access, so two wall screens can show different pages. The same app and settings in both ways of running the edge client.